

Starmap Project — Technical Development and Pipeline Documentation

1. Project Overview

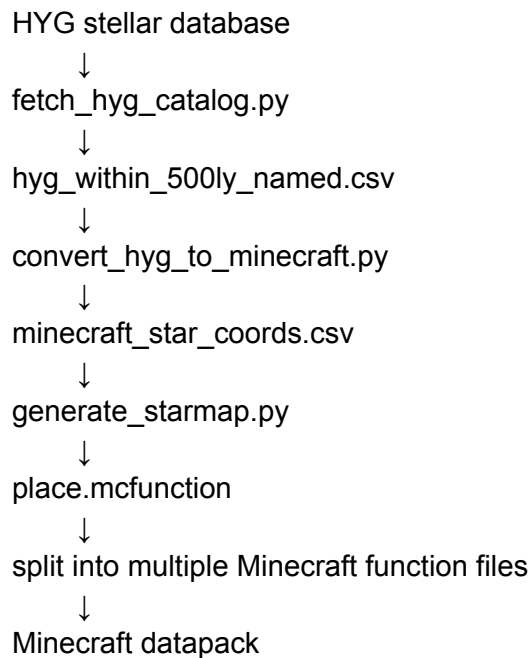
The Starmap project is a Python-based astronomical visualization pipeline that converts real stellar catalog data into a three-dimensional Minecraft star map.

The goal was to take real stars within approximately **500 light-years of the Sun**, convert their astronomical positions into Minecraft-compatible Cartesian coordinates, visually encode basic stellar properties using Minecraft blocks and shapes, and finally generate Minecraft `.mcf` files capable of constructing the map automatically.

The finished project involved three main stages:

1. **Acquire and clean astronomical data**
2. **Convert astronomical coordinates and stellar properties into Minecraft representations**
3. **Generate Minecraft commands and datapack functions**

The basic pipeline was:



The project went through several experimental approaches before reaching this final design. In particular, Gaia data and armor-stand-based labels were tested but ultimately abandoned because they created practical problems.

2. Final Project Structure

A clean version of the repository can be organized as:

```
stellar-mapping-project/
├── README.md
├── requirements.txt
├── .gitignore
├── src/
│   ├── fetch_hyg_catalog.py
│   ├── convert_hyg_to_minecraft.py
│   └── generate_starmap.py
├── data/
│   ├── hyg_within_500ly_named.csv
│   └── minecraft_star_coords.csv
├── datapack/
│   └── ...
├── screenshots/
│   └── ...
└── docs/
    └── project_documentation.md
```

The names shown above are cleaned-up professional names for the original scripts.

The original working filenames were:

```
fetch hipparcos dataset.py
Convert HYG to MCcoords.py
Setblock MC.py
```

The recommended GitHub names are:

```
fetch_hyg_catalog.py  
convert_hyg_to_minecraft.py  
generate_starmap.py
```

Python filenames are normally written in lowercase with underscores and without spaces.

3. Dataset Choice

Initial Gaia Experiment

The project initially experimented with the **Gaia stellar catalog**.

A separate script was created to query SIMBAD using Gaia right ascension and declination coordinates and retrieve astronomical identifiers.

That experimental pipeline looked approximately like:

```
gaia_within_500ly.csv  
↓  
SIMBAD query script  
↓  
stars_named.csv
```

The SIMBAD script searched within five arcseconds of each Gaia position and attempted to retrieve the object's **MAIN_ID**.

However, the Gaia-based approach caused too many practical problems and was not used in the final project.

The surviving script named:

```
SIMBAD Names.py
```

belongs to this abandoned Gaia experiment rather than the final Starmap pipeline.

The final project instead used the **HYG Database**.

4. HYG Database

The final project used the HYG stellar database as its primary source.

The first production script downloads:

hygdata_v41.csv

from the HYG Database repository.

HYG combines useful stellar information from several astronomical catalogs and contains fields including:

- HYG identifier
- Hipparcos identifier
- Henry Draper identifier
- Harvard Revised identifier
- Gliese identifier
- Bayer/Flamsteed designation
- proper name
- right ascension
- declination
- distance
- apparent magnitude
- spectral classification

This made HYG particularly useful for the project because both positional information and basic stellar metadata were available in one dataset.

5. Stage 1 — Fetching and Preparing the Stellar Dataset

Script

Original filename:

fetch hipparcos dataset.py

Recommended filename:

fetch_hyg_catalog.py

Despite the original filename, this script does not directly download the Hipparcos catalog. It downloads the HYG v4.1 dataset.

Its primary output is:

hyg_within_500ly_named.csv

5.1 Downloading the Catalog

The script contains a URL pointing to:

hygdata_v41.csv

It downloads the catalog using Python's `requests` library.

A basic caching system is also included. If a sufficiently large local copy of the HYG CSV already exists, the script uses that copy instead of downloading the entire dataset again.

This avoids repeatedly downloading a large astronomical catalog during development.

5.2 Filtering the Map Radius

The Starmap represents stars within:

500 light-years

The HYG dataset stores distance primarily in parsecs, so the program first converts the 500-light-year radius into parsecs:

$R_{PC} = 500 / 3.26156$

It then filters out stars farther away than that distance.

The script also removes entries without usable distance values.

5.3 Right Ascension Conversion

HYG stores right ascension in hours.

The program converts right ascension to degrees using:

RA degrees = RA hours $\times$ 15

The resulting columns are stored as:

RA_deg
DEC_deg

Declination was already suitable for conversion into degrees.

5.4 Distance Columns

The program retains both parsec and light-year distances:

distance_pc
distance_ly

with:

distance_ly = distance_pc $\times$ 3.26156

This made the later coordinate-conversion stage easier to understand because the map scale was conceptually based on light-years.

6. Stellar Naming System

One important part of the preprocessing stage was making sure every star had some usable label.

The program builds an alias field using identifiers including:

HIP
HD
HR
Gliese
Bayer/Flamsteed

It also creates a final `label` column.

The naming priority is approximately:

Proper name
↓ if unavailable
Bayer / Flamsteed designation
↓ if unavailable
HIP identifier
↓ if unavailable
HYG identifier

Therefore, a famous star might appear using its common proper name, while a less well-known object could instead appear as something like:

HIP 12345

or:

HYG 54321

The script explicitly implements these fallback steps.

7. Output of Stage 1

The first script creates:

hyg_within_500ly_named.csv

The important columns include:

id
hip
label
proper
aliases
RA_deg
DEC_deg
distance_pc
distance_ly
spect
mag

This CSV acts as the cleaned astronomical input for the coordinate-conversion stage.

8. Stage 2 — Converting HYG Coordinates into Minecraft Coordinates

Script

Original filename:

Convert HYG to MCcoords.py

Recommended filename:

convert_hyg_to_minecraft.py

Input:

hyg_within_500ly_named.csv

Output:

minecraft_star_coords.csv

The script explicitly defines these two files as its input and output.

This script performs two major jobs:

1. Convert astronomical coordinates into Minecraft XYZ coordinates.
2. Convert stellar classifications into Minecraft blocks and shapes.

9. Spherical Astronomical Coordinates to Cartesian Coordinates

Astronomical locations were provided using:

- Right Ascension
- Declination
- Distance

Minecraft, however, requires:

X
Y
Z

coordinates.

The program converts the astronomical spherical coordinates into a Cartesian vector using:

$$x = d \cos(\text{dec}) \cos(\text{ra})$$

$$y = d \cos(\text{dec}) \sin(\text{ra})$$

$$z = d \sin(\text{dec})$$

where:

d = distance in light-years

ra = right ascension

dec = declination

This conversion is directly implemented in the coordinate script.

The resulting vector represents the star's three-dimensional location relative to the Sun.

10. Coordinate-System Rotation

The raw right-ascension/declination coordinate system was not the desired orientation for the finished Minecraft map.

A rotation matrix was therefore applied:

R_matrix

The raw star vector is multiplied by this matrix:

$v_finish = R_matrix @ v_raw$

This rotates the stellar coordinates into the orientation used by the finished map.

11. Mapping the Axes into Minecraft

After rotation, the values are rounded to integer Minecraft block coordinates.

The script then remaps the axes:

Minecraft X = transformed X

Minecraft Y = transformed Z

Minecraft Z = transformed Y

This occurs in the script as:

$X_{in}, Y_{pro}, Z_{up} = v_block$

$mc_x = X_{in}$

```
mc_y = Zup  
mc_z = Ypro
```

The conversion scale was left at:

SCALE = 1.0

meaning the stellar distances were not intentionally compressed at this stage.

Approximately one light-year therefore corresponds to roughly one block before rounding.

12. Representing Stellar Spectral Classes

The project uses different Minecraft blocks to approximately distinguish stellar spectral classes.

The conversion script contains the following mapping:

Spectral Class	Minecraft Representation
M	Redstone Block
K	Shroomlight
G	Glowstone
F	Ochre Froglight
A	Pearlescent Froglight
B	Sea Lantern
White Dwarf / D	Blue Ice
Unknown or unmatched	Amethyst Block

This visual encoding allows the viewer to recognize approximate stellar type directly from the Minecraft map.

For example:

M-class red dwarf



Redstone Block

while a Sun-like G-class star becomes:

G-class star



Glowstone

The system is primarily a visual classification scheme rather than an attempt to reproduce exact stellar colors.

13. Representing Luminosity Class and Stellar Size

The program also parses the luminosity class contained in the spectral classification.

The following classes are recognized:

Ia
Ib
II
III
IV
V

The conversion script assigns three Minecraft shape categories:

single
star_cross
cube_3

The mapping is:

Luminosity Class	Approximate Star Type	Minecraft Shape
V	Main-sequence / dwarf	Single block

IV	Subgiant	7-block cross
III	Giant	7-block cross
II	Bright giant	7-block cross
Ib	Supergiant	3×3×3 cube
Ia	Luminous supergiant	3×3×3 cube

The `star_cross` consists of:

block

block - center - block

block

with additional blocks extending forward and backward in the Z direction.

In total:

1 center + 6 surrounding blocks = 7 blocks

The `cube_3` representation is a complete:

$3 \times 3 \times 3$

cube containing 27 blocks.

This allows especially luminous stars to appear physically larger in the map.

14. Output of Stage 2

The coordinate conversion script creates:

`minecraft_star_coords.csv`

Its columns include:

x
y
z
block
size_category
label
proper
aliases
hip
spect
magnitude

At this stage the astronomical calculations are essentially complete.

Each row now describes exactly how one star should appear inside Minecraft.

For example, conceptually:

x = 23
y = -12
z = 47

block = minecraft:redstone_block
size_category = single

label = Proxima Centauri
spect = M5.5V

The remaining problem is simply converting those rows into Minecraft commands.

15. Stage 3 — Generating Minecraft Commands

Script

Original filename:

Setblock MC.py

Recommended filename:

`generate_starmap.py`

Input:

`minecraft_star_coords.csv`

Output:

`place.mcfuction`

The script explicitly reads `minecraft_star_coords.csv` and generates Minecraft commands from it.

16. Minecraft Vertical Offset

A vertical offset of:

136 blocks

is added to the calculated Y coordinate.

The script performs:

$y = \text{original_y} + 136$

This places the Starmap around a suitable altitude inside the Minecraft world rather than allowing much of the map to fall below the normal world area.

The project treated the Solar position as being approximately centered around:

(0, 136, 0)

in Minecraft.

17. Placing Stars

For every row of `minecraft_star_coords.csv`, the generator reads:

```
x  
y  
z  
block  
size_category  
label  
magnitude
```

It then converts the `size_category` into one or more Minecraft `setblock` commands.

A main-sequence star may generate:

```
setblock X Y Z minecraft:glowstone replace
```

A giant generates seven `setblock` commands corresponding to the star-cross geometry.

A supergiant can generate 27 commands corresponding to the full 3×3×3 cube.

The generator therefore turns the abstract CSV representation into actual commands Minecraft can execute.

18. Handling Coordinate Collisions

Because astronomical coordinates eventually have to be rounded to whole Minecraft blocks, multiple stars can occasionally round to the same position.

The generator contains a simple collision-handling system.

It keeps track of already occupied positions and offsets additional objects sideways when required.

In the simpler final generator, this appears as:

$x \neq k$

for repeated coordinates.

This prevents multiple stars from being completely hidden inside one another after coordinate rounding.

19. Star Labels

One major technical problem was how to display stellar names without causing severe Minecraft performance problems.

Early Approach — Armor Stands

An earlier experimental system used invisible armor stands positioned at each star.

The armor stands stored custom names and tags.

A nearby-label function could then search for armor stands within a given radius and create visible text displays near the player.

The experimental `Setblock MC2.py` explicitly uses:

```
LABEL_RADIUS = 15
```

and creates invisible armor stands tagged as:

```
starmap_star
```

It then generated a separate:

```
labels_near.mcfuction
```

that found armor stands within 15 blocks and spawned text labels near them.

Although functional, the armor-stand approach was ultimately undesirable because a very large number of entities caused excessive Minecraft lag.

20. Final Label Approach — Text Display Entities

The final simpler generator abandoned the armor-stand dependency.

Instead, each labeled star directly receives a Minecraft:

`text_display`

`entity`.

The command generated resembles:

```
summon text_display X Y Z {  
  text: ...,  
  billboard:"center",  
  view_range:0.23f,  
  scale:[0.7f,0.7f,0.7f]  
}
```

The label is positioned:

2 blocks above the star

and the billboard setting keeps the text facing the viewer.

21. Approximately 15-Block Label Visibility

A major optimization was preventing every stellar label from being visible at all times.

The final generator uses:

`view_range:0.23f`

Minecraft text-display view range is scaled relative to the game's entity rendering distance, making this value correspond to roughly:

15 blocks

for the intended setup.

Therefore, labels appear only when the player approaches a star.

This solved two problems:

1. The screen is not overwhelmed by thousands of overlapping stellar names.
2. Minecraft does not need to visibly render every label across the entire map at once.

The intended visual experience is therefore:

Far from star



Only star block visible

Approach within ~15 blocks



Star name becomes visible

22. Why the Final `.mcfunction` Had to Be Split

The Python generator initially writes every generated command into one file:

`place.mcfunction`

The script ends by joining all commands and writing them to that file.

However, the Starmap contained tens of thousands of stellar objects.

Each star could require:

- one or more block-placement commands
- one label command

- several additional block commands for giants and supergiants

The resulting `.mcfuction` became too large to conveniently execute as one Minecraft function.

The practical solution used in the finished datapack was therefore to divide the generated commands into approximately ten separate function files.

Conceptually:

place.mcfuction



split into chunks



place_01.mcfuction
place_02.mcfuction
place_03.mcfuction
place_04.mcfuction
place_05.mcfuction
place_06.mcfuction
place_07.mcfuction
place_08.mcfuction
place_09.mcfuction
place_10.mcfuction

These functions were then executed separately inside Minecraft.

A future improvement would be to modify `generate_starmap.py` so that it automatically generates the ten chunks instead of requiring the oversized function to be split afterward.

Another useful improvement is a wrapper Minecraft function that calls the individual chunks, allowing the entire Starmap generation process to be initiated through one command.

23. Experimental Scripts

Several scripts were created while testing different approaches.

These are useful historically but are not required for the final pipeline.

Examples include:

SIMBAD Names.py
Setblock MC2.py
Setblock MC3.py

SIMBAD Names.py

This belongs to the abandoned Gaia-based data-processing experiment.

It:

reads gaia_within_500ly.csv
↓
queries SIMBAD
↓
adds MAIN_ID identifiers
↓
writes stars_named.csv

It is not part of the final HYG-based workflow.

Setblock MC2.py

This version experimented with invisible armor stands as persistent stellar markers and generated a separate nearby-label function.

It explicitly used a 15-block radius for nearby labels.

Setblock MC3.py

This version combined:

armor_stand markers
+
text_display entities

and assigned the text displays a view range equivalent to approximately 15 blocks.

Because the finished approach did not require an armor stand for every star, these versions are better treated as development experiments rather than production scripts.

For the public GitHub repository, the main `src/` folder should contain only the scripts required for the final pipeline.

Old experiments can either remain offline or be placed in a clearly labeled archive directory.

24. Final Production Pipeline

The final project can be understood as three major transformations.

Step 1 — Astronomical Catalog → Filtered Stellar Dataset

HYG Database v4.1



`fetch_hyg_catalog.py`



Filter to stars ≤ 500 light-years
Convert RA hours → degrees
Generate names and aliases
Preserve spectral classifications



`hyg_within_500ly_named.csv`

Step 2 — Stellar Dataset → Minecraft Coordinate Dataset

`hyg_within_500ly_named.csv`



`convert_hyg_to_minecraft.py`



RA / Dec / distance



3D Cartesian coordinates



coordinate rotation



Minecraft axis mapping

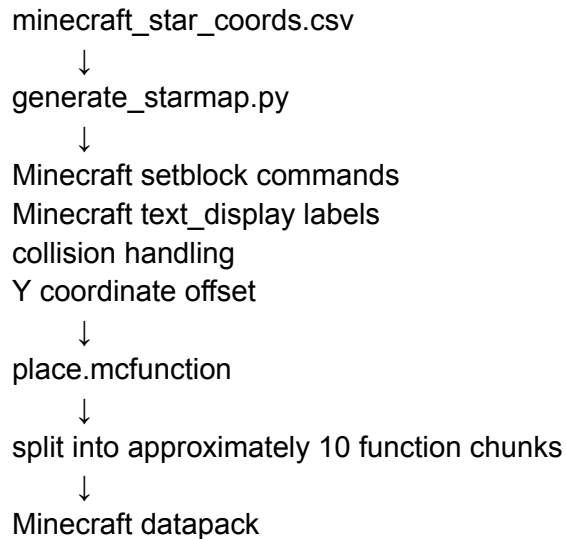


spectral class → block type
luminosity class → star shape



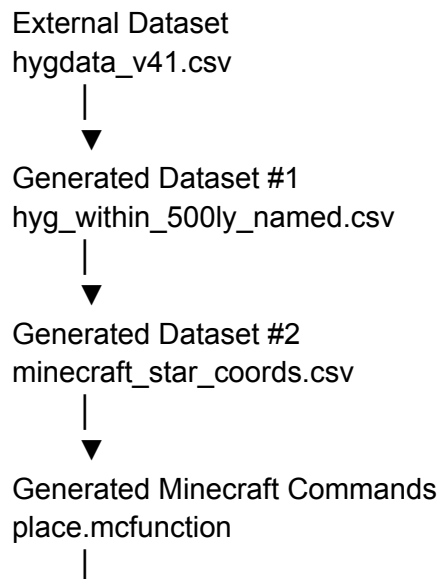
`minecraft_star_coords.csv`

Step 3 — Minecraft Dataset → Minecraft Datapack Commands



25. Data Dependency Chain

One important feature of the project is that each generated dataset becomes the input of the next processing stage.





Final Datapack Function Files

place_01.mcfunction ... place_10.mcfunction

Therefore:

hygdata_v41.csv

Role:

Raw external astronomical dataset

hyg_within_500ly_named.csv

Role:

Cleaned and filtered astronomical dataset

Produced by:

fetch_hyg_catalog.py

Used by:

convert_hyg_to_minecraft.py

minecraft_star_coords.csv

Role:

Minecraft-ready stellar dataset

Produced by:

convert_hyg_to_minecraft.py

Used by:

generate_starmap.py

.mcfunction files

Role:

Executable Minecraft construction commands

Produced from:

minecraft_star_coords.csv

and used directly by the Minecraft datapack.

26. Software Used

The project primarily used Python.

Important Python libraries included:

pandas
numpy
requests
math
re
pathlib

pandas

Used for:

- loading CSV datasets
- filtering rows
- creating derived columns
- exporting processed datasets

NumPy

Used primarily for:

- vectors
- matrix multiplication
- coordinate rotation
- coordinate rounding

requests

Used to download the HYG stellar catalog.

math

Used for:

- sine
- cosine
- degree-to-radian conversion
- Cartesian coordinate calculation

pathlib

Used for cleaner file and directory paths.

re

Used for parsing spectral classifications.

27. Major Technical Problems Encountered

The project required solving several practical problems beyond simply converting coordinates.

Problem 1 — Dataset Selection

Gaia initially appeared attractive because of its enormous precision and size.

However, integrating Gaia data into the desired Minecraft workflow introduced substantial difficulties, especially around naming, filtering, and practical data handling.

The project therefore switched to HYG, which provided a more convenient combination of:

- coordinate information
 - stellar distances
 - spectral classifications
 - common identifiers
 - proper names
-

Problem 2 — Astronomical Coordinates Are Not Minecraft Coordinates

RA and declination describe angular positions on the celestial sphere rather than ordinary XYZ coordinates.

They therefore had to be mathematically converted using spherical-to-Cartesian equations.

The resulting coordinate system also had to be rotated and remapped to Minecraft's axes.

Problem 3 — Different Stars Needed Meaningful Visual Representation

Simply placing identical blocks would lose most of the useful astronomical information.

The solution was to encode:

spectral class → block material

luminosity class → shape / size

This allows the map to communicate some stellar properties visually.

Problem 4 — Multiple Stars Can Occupy the Same Minecraft Block

Real stellar positions are continuous.

Minecraft block coordinates are discrete integers.

Rounding therefore creates occasional collisions.

A positional offset system was introduced so overlapping stars could still be represented separately.

Problem 5 — Stellar Labels Caused Performance Problems

Using armor stands for stellar labels created excessive entity overhead.

The final approach used Minecraft `text_display` entities instead.

Their rendering range was restricted to approximately 15 blocks so labels only became visible when needed.

Problem 6 — The Generated Function Was Too Large

The complete command output was too large to comfortably execute as one `.mcfuction`.

The practical solution was to split the final command output into roughly ten separate function files and execute those sequentially.

28. Why the Project Is Useful

Although Minecraft is the visualization environment, the project involves several broader technical concepts:

- astronomical data processing
- coordinate-system transformations
- spherical geometry
- matrix transformations
- procedural generation
- data cleaning
- classification parsing
- large-dataset processing
- visualization design

- performance optimization
- automated command generation

Minecraft serves as an interactive three-dimensional environment in which the transformed astronomical dataset can be explored spatially.

The project therefore demonstrates the complete process of turning a scientific dataset into an interactive visualization.

29. Recommended Final Repository Files

For the public GitHub repository, the main project files should be approximately:

```
stellar-mapping-project/
|
|— README.md
|— requirements.txt
|— .gitignore
|
|— src/
|   |— fetch_hyg_catalog.py
|   |— convert_hyg_to_minecraft.py
|   |— generate_starmap.py
|
|— data/
|   |— hyg_within_500ly_named.csv
|   |— minecraft_star_coords.csv
|
|— datapack/
|   |— ...
|
|— screenshots/
|   |— ...
|
|— docs/
|   |— project_documentation.md
```

Experimental code such as Gaia scripts and abandoned armor-stand implementations does not need to remain in the main `src/` directory.

If preserved publicly, it should instead be separated into something like:

archive/

or:

experiments/

so that readers can immediately distinguish production code from developmental experiments.

30. Summary

The Starmap project converts real nearby stellar data into a Minecraft-scale three-dimensional visualization.

The final workflow is:

```
graph TD; A[HYG stellar catalog] --> B[filter and clean nearby stars]; B --> C[assign usable names and identifiers]; C --> D[convert RA / Dec / distance to XYZ]; D --> E[rotate and remap coordinates]; E --> F[assign Minecraft blocks by spectral type]; F --> G[assign shapes by luminosity class]; G --> H[produce Minecraft-ready CSV]; H --> I[generate star blocks and nearby text labels]; I --> J[split generated commands into manageable functions]; J --> K[load and execute as a Minecraft datapack];
```

The final core scripts are:

fetch_hyg_catalog.py
convert_hyg_to_minecraft.py
generate_starmap.py

The principal intermediate datasets are:

hyg_within_500ly_named.csv
minecraft_star_coords.csv

The final Minecraft representation uses:

- block material to indicate spectral class
- marker size and shape to indicate luminosity class
- `text_display` entities for stellar labels
- approximately 15-block label visibility
- separate `.mcfunction` chunks to handle the large number of generated commands

The result is an interactive Minecraft representation of tens of thousands of real nearby stars built from astronomical catalog data through a reproducible Python processing pipeline.